

Cloud AI Internship Practice Project

AI-Powered Vehicle Incident Analyzer

A beginner-to-intermediate project inspired by the internship role at AUDI AG.

This project helps you practice:

- Python development
- FastAPI APIs
- Pydantic validation
- AWS cloud deployment concepts
- Docker
- Infrastructure as Code (AWS CDK)
- Testing with pytest
- Simple AI pipeline integration

1. Project Idea

Build a backend system where:

- Users upload a vehicle incident description
- The API analyzes the text using a simple AI model
- The system predicts:
 - Incident category
 - Severity
 - Recommended action
- Results are stored and exposed through APIs
- Entire app is containerized with Docker
- Deployment-ready for AWS

This is realistic for automotive cloud engineering + AI workflows.

2. Final Architecture

Frontend (optional later)

|

v

FastAPI Backend

|

```
+----- AI Service (simple ML/NLP logic)
|
+----- PostgreSQL / SQLite
|
+----- Logging
|
+----- Tests (pytest)
|
+----- Docker Container
|
+----- AWS Deployment (future)
      |
      +----- ECS or Lambda
      +----- API Gateway
      +----- CloudWatch
```

3. Technologies You Will Learn

Area	Technology
Backend API	FastAPI
Validation	Pydantic
Programming	Python
Testing	pytest
Containers	Docker
Cloud	AWS
IaC	AWS CDK
AI	scikit-learn / transformers later
Database	SQLite/PostgreSQL

4. Beginner-Friendly MVP

Start SIMPLE.

Do NOT begin with AWS immediately.

Phase 1 goal:

Build a local FastAPI app that:

- Accepts incident reports
- Classifies them
- Returns AI analysis
- Saves data

Example Input:

```
{
  "vehicle_id": "AUDI-A8-1001",
  "incident_description": "Engine overheating warning appeared during highway driving"
}
```

Example Output:

```
{
  "category": "Engine",
  "severity": "High",
  "recommended_action": "Immediate inspection required"
}
```

5. Folder Structure

```
vehicle-ai-platform/
|
├─ app/
|   ├── main.py
|   ├── routes/
|   │   └─ incidents.py
|   ├── services/
|   │   └─ ai_service.py
|   ├── models/
|   │   └─ schemas.py
|   ├── database/
|   │   └─ db.py
|   └─ tests/
|       └─ test_api.py
|
├─ Dockerfile
└─ requirements.txt
```

```
|— README.md
|— infrastructure/
|   |— cdk/
```

6. Step-by-Step Roadmap

PHASE 1 — Python Foundations

Before coding project features, become comfortable with:

Learn:

- Variables
- Functions
- Classes
- Lists/dictionaries
- File handling
- Virtual environments
- pip
- Exception handling

Practice:

- Build small Python scripts
- Parse JSON
- Read/write files
- Build simple classes

7. PHASE 2 — FastAPI Basics

Install

```
pip install fastapi uvicorn
```

Minimal API

```
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/")
def home():
    return {"message": "Vehicle AI Platform Running"}
```

Run:

```
uvicorn app.main:app --reload
```

8. PHASE 3 — Pydantic Models

This is IMPORTANT for the internship.

schemas.py

```
from pydantic import BaseModel

class IncidentRequest(BaseModel):
    vehicle_id: str
    incident_description: str
```

Why this matters:

- Validation
- Clean APIs
- Production-quality backend design

9. PHASE 4 — Build AI Logic

Initially keep AI simple.

ai_service.py

```
def analyze_incident(description: str):
    text = description.lower()

    if "engine" in text:
        return {
            "category": "Engine",
```

```

        "severity": "High",
        "recommended_action": "Immediate inspection required"
    }

    if "brake" in text:
        return {
            "category": "Brake",
            "severity": "Critical",
            "recommended_action": "Stop vehicle safely"
        }

    return {
        "category": "General",
        "severity": "Medium",
        "recommended_action": "Schedule service"
    }

```

This is enough initially.

Later you can replace it with:

- scikit-learn
- Hugging Face transformers
- OpenAI APIs
- AWS Bedrock

10. PHASE 5 — API Endpoint

incidents.py

```

from fastapi import APIRouter
from app.models.schemas import IncidentRequest
from app.services.ai_service import analyze_incident

router = APIRouter()

@router.post("/analyze")
def analyze(data: IncidentRequest):
    result = analyze_incident(data.incident_description)
    return result

```

11. PHASE 6 — Testing with pytest

This is heavily valued in industry.

Install:

```
pip install pytest
```

Example test:

```
from app.services.ai_service import analyze_incident

def test_engine_incident():
    result = analyze_incident("engine overheating")

    assert result["category"] == "Engine"
```

Learn:

- Assertions
 - Test cases
 - API testing
 - Edge cases
-

12. PHASE 7 — Docker

This is VERY important for cloud engineering.

Dockerfile

```
FROM python:3.11

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Build:

```
docker build -t vehicle-ai-platform .
```

Run:

```
docker run -p 8000:8000 vehicle-ai-platform
```

13. PHASE 8 — AWS Basics

After local project works.

Learn these AWS services:

Service	Purpose
EC2	Virtual machine
ECS	Container deployment
S3	File storage
IAM	Permissions
CloudWatch	Logs/Monitoring
API Gateway	API management
Lambda	Serverless

14. PHASE 9 — Infrastructure as Code (CDK)

The job description explicitly mentions IaC.

Learn AWS CDK

Install:


```
npm install -g aws-cdk
```

Example CDK idea:

```
from aws_cdk import Stack
```

Goal:

- Deploy infrastructure using code
- Avoid manual AWS setup

This is a major interview topic.

15. PHASE 10 — Advanced Features

After MVP:

Add:

Authentication

- JWT
- User login

Database

- PostgreSQL
- SQLAlchemy

AI Improvements

- Sentiment analysis
- Real ML model
- Embeddings
- Vector DB

CI/CD

- GitHub Actions
- Automated testing
- Automated deployment

Monitoring

- Logging
 - Metrics
 - Error tracking
-

16. Interview Preparation Topics

You should be able to explain:

Python

- OOP
- Async programming
- Exception handling
- Virtual environments

FastAPI

- Routing
- Dependency injection
- Pydantic
- Validation

Docker

- Images
- Containers
- Ports
- Dockerfile

AWS

- Why cloud?
- Scalability
- IAM basics
- ECS vs Lambda

Testing

- Unit testing
- pytest
- Mocking

AI

- What is an AI pipeline?
 - Data preprocessing
 - Model inference
 - API integration
-

17. GitHub Portfolio Tips

Your GitHub should contain:

- Clean README
- Architecture diagram
- Docker instructions
- Screenshots
- API documentation
- Tests
- Commit history

Good README sections:

- Overview
 - Features
 - Tech Stack
 - Setup
 - API Endpoints
 - Docker Setup
 - Future Improvements
-

18. What Interviewers May Ask

Python

- Difference between list and tuple?
- What is a decorator?
- What is async/await?

FastAPI

- Why FastAPI over Flask?
- What is Pydantic?

- How validation works?

Docker

- Difference between image and container?
- Why containers?

AWS

- What is EC2?
- What is IAM?
- What is Infrastructure as Code?

AI

- How would you deploy an ML model?
 - What is an inference pipeline?
-

19. Your Best Learning Sequence

Do NOT try learning everything simultaneously.

Recommended order:

1. Python fundamentals
 2. APIs with FastAPI
 3. Pydantic
 4. pytest
 5. Docker
 6. Git/GitHub
 7. AWS basics
 8. AWS CDK
 9. AI integrations
 10. CI/CD
-

20. 30-Day Beginner Plan

Week 1

- Python basics
- Functions
- OOP

- JSON
- Virtual environments

Week 2

- FastAPI
- Pydantic
- API routes
- Swagger docs

Week 3

- pytest
- Docker
- GitHub polishing

Week 4

- AWS basics
- ECS concepts
- CDK introduction
- Deploy mini app

21. Final Goal

By the end, you should be able to confidently say:

"I built and containerized a FastAPI-based AI application, tested it with pytest, and prepared it for AWS deployment using Infrastructure-as-Code principles."

That sentence alone aligns strongly with the internship description.